arXiv:2208.02235v2 [cs.LG] 10 Aug 2022

QUANTUM-INSPIRED TENSOR NEURAL NETWORKS FOR PARTIAL DIFFERENTIAL EQUATIONS

A PREPRINT

*Raj G. Patel[1,4], Chia-Wei Hsing[2], Serkan Şahin[2], Saeed S. Jahromi[2,5], Samuel Palmer[1], Shivam Sharma[2], Christophe Michel[3], Vincent Porte[3], Mustafa Abid[3], Stéphane Aubert[3], Pierre Castellani[3], Chi-Guhn Lee[4], Samuel Mugel[1], and Román Orús[2,5,6]

[1]Multiverse Computing, Centre for Social Innovation, 192 Spadina Ave, Suite 509, Toronto, M5T 2C2, Canada
[2]Multiverse Computing, Paseo de Miramón 170, 20014 San Sebastián, Spain
[3]Crédit Agricole, 12, Place des Etats-Unis - CS 70052 - 92547 Montrouge Cedex, France
[4]University of Toronto, Toronto, Ontario M5S 2E4, Canada
[5]Donostia International Physics Center, Paseo Manuel de Lardizabal 4, E-20018 San Sebastián, Spain
[6]Ikerbasque Foundation for Science, Maria Diaz de Haro 3, E-48013 Bilbao, Spain

August 11, 2022

ABSTRACT

Partial Differential Equations (PDEs) are used to model a variety of dynamical systems in science and engineering. Recent advances in deep learning have enabled us to solve them in a higher dimension by addressing the curse of dimensionality in new ways. However, deep learning methods are constrained by training time and memory. To tackle these shortcomings, we implement Tensor Neural Networks (TNN), a quantum-inspired neural network architecture that leverages Tensor Network ideas to improve upon deep learning approaches. We demonstrate that TNN provide significant parameter savings while attaining the same accuracy as compared to the classical Dense Neural Network (DNN). In addition, we also show how TNN can be trained faster than DNN for the same accuracy. We benchmark TNN by applying them to solve parabolic PDEs, specifically the Black-Scholes-Barenblatt equation, widely used in financial pricing theory, empirically showing the advantages of TNN over DNN. Further examples, such as the Hamilton-Jacobi-Bellman equation, are also discussed.

1 Introduction

Partial Differential Equations (PDEs) are an indispensable tool for modeling a variety of problems ranging from physics to finance. Typical approaches for solving such PDEs mostly rely on classical mesh-based numerical methods or Monte Carlo approaches. However, scaling these to higher dimensions has always been a challenge because of their dependency on spatio-temporal grids as well as on a large number of sample paths. As an alternative, recent advancements in deep learning have enabled us to circumvent some of these challenges by approximating the unknown solution using Dense Neural Networks (DNNs) [23, 7, 16, 30]. The basic idea of these approaches is to leverage the connection between high-dimensional PDEs and forward-backward stochastic differential equations (FBSDE) [6]. The solution of the corresponding FBSDE can be written as a deterministic function of time and the state process. Under suitable regularity assumptions, the FBSDE solution can represent the solution of the underlying parabolic PDE. Efficient methods for approximating the FBSDE solution with a DNN have been put forward recently in Refs. [24, 25]. However, in spite of their apparent success, DNN approaches for solving PDEs are computationally expensive and limited by memory [19, 27, 32].

*Email: raj.patel@multiversecomputing.com

In this paper we show how to overcome this problem by combining Tensor Networks (TN) [20] with DNNs. TNs were proposed in physics to efficiently describe strongly-correlated structures, such as quantum many-body states of matter. They are at the basis of well-known numerical simulation methods, such as Density Matrix Renormalization Group (DMRG) [31], Time-Evolving Block Decimation (TEBD) [29], and more. At the fundamental level, TNs are nothing but efficient descriptions of high-dimensional vectors and operators. Because of this, TNs have recently found important applications also in different domains of computer science such as machine learning (ML) and optimization. In particular, TNs have proven successful in ML tasks such as classification [11, 10, 13, 12, 3, 17, 14], generative modeling [15, 5, 28] and sequence modeling [4].

In practice, we use a specific type of TN known as Matrix Product Operator (MPO) (also known as Tensor Train as used in Ref. [19]), which generalizes the low-rank idea by representing the weight matrix in terms of an efficient TN description. Following ideas from Refs. [19, 21], we transform a DNN into what we call a Tensor Neural Network (TNN), in turn enhancing training performance and reducing memory consumption. Our approach saves memory since it uses fewer parameters compared to a standard Neural Network (NN) [19]. While low-rank representations of the weight matrices had already been discussed in the literature [26, 32, 9], we certify the improvement obtained by TNN by showing that we cannot find an equally-performing DNN with the same number of parameters as the TNN. We achieve this by analyzing the entire sample space of DNN architectures with equivalent number of parameters as that of a TNN. To the best of our knowledge, this important milestone has not yet been considered in the literature. Our main test bed for benchmark is the Black-Scholes-Barenblatt equation, widely used in financial pricing theory.

This paper is organized as follows: in Section 2, we show how a parabolic PDE can essentially be mapped to an SDE and how the corresponding SDE can be solved with a neural network. In Section 3, we briefly review the concept of TN and show how one can incorporate them in NN to obtain TNN. In Section 4 we demonstrate how TNN can outperform DNN for the case of Black-Scholes-Barenblatt PDE. Finally, in Section 5 we present our conclusions. Furthermore, Appendix A discusses mathematical details of the Feynman-Kac representation, and Appendix B shows further experiments, including benchmarks for the Hamilton-Jacobi-Bellman equation.

2 Neural Networks for PDE

The connection between PDE and Forward Backward Stochastic Differential Equations (FBSDE) is well studied in Refs. [6, 2, 18, 22, 8]. For $t \in [0, T]$ consider the following system of Stochastic Differential Equations (SDE),

$$\begin{aligned} dX_t &= \mu(t, X_t, Y_t, Z_t)dt + \sigma(t, X_t, Y_t)dW_t \\ dY_t &= \varphi(t, X_t, Y_t, Z_t)dt + Z_t^T \sigma(t, X_t, Y_t)dW_t, \end{aligned} \tag{1}$$

where $X_0 = \epsilon$ is the initial condition for the forward SDE whereas $Y_T = g(X_T)$ is the terminal condition for the backward SDE with known $g : \mathbb{R}^d \to \mathbb{R}$. Furthermore, W_t is the vector valued Brownian motion. A solution to this set of equations would have X_t, Y_t, Z_t.

Now, consider the non-linear PDE

$$u_t = f(t, x, u, \mathcal{D}u, \mathcal{D}^2 u), \tag{2}$$

where $\mathcal{D}u, \mathcal{D}^2 u$ represents gradient and Hessian of u respectively whereas function f is given by

$$f(t, x, y, z, \gamma) = \varphi(t, x, y, z) - \mu(t, x, y, z)^T z - \frac{1}{2}\mathrm{Tr}\left(\sigma(t, x, y)\sigma(t, x, y)^T \gamma\right). \tag{3}$$

For the given function f and terminal condition $u(T, x) = g(x)$, it is known that $Y_t = u(t, X_t)$ and $Z_t = \nabla u(t, X_t)$ by Ito's Lemma. This is how the SDEs in Eq.(1) are related to the PDE in Eq.(2), so that if we know the solution to one, we can find or approximate the other.

Now, if the solution to the PDE in Eq.(2) is known, then we can easily approximate the solution to SDEs in Eq.(1) by leveraging Euler-Maruyama discretization of SDEs in Eq.(1) such that:

$$\begin{aligned} \Delta W_n &\sim \mathcal{N}(0, \Delta t_n) \\ X_{n+1} - X_n &\approx \mu(t_n, X_n, Y_n, Z_n)\Delta t_n + \sigma(t_n, X_n, Y_n, Z_n)\Delta W_n \\ Y_{n+1} - Y_n &\approx \varphi(t_n, X_n, Y_n, Z_n)\Delta t_n + Z_n^T \sigma(t_n, X_n, Y_n, Z_n)\Delta W_n. \end{aligned} \tag{4}$$

where $Y_n = u(t_n, X_n)$ and $Z_n = \nabla u(t_n, X_n)$. However, as the solution of the PDE is not known, we parameterize the solution u_t using a neural network with parameters θ as proposed in Ref. [23]. We obtain the required gradient vector

using automatic differentiation. Parameters of the neural network representing $u(t, x)$ can be learned by minimizing the following loss function obtained by discretizing the SDE as done in Eq.(4):

$$\min_{\theta}\Bigg(\sum_{m=1}^{M}\sum_{n=0}^{N-1}\big|Y_{n+1}^{m}(\theta)-Y_{n}^{m}(\theta)-\varphi(t_n,X_n^m,Y_n^m(\theta),Z_n^m(\theta))\Delta t_n \\ -(Z_n^m(\theta))^T\sigma(t_n,X_n^m,Y_n^m(\theta))\Delta W_n^m\big|^2+\sum_{m=1}^{M}(Y_N^m(\theta)-g(X_N^m))^2\Bigg). \quad (5)$$

where M is the batch size, m labels the m-th realization of the underlying Brownian motion, and n labels time t_n.

The goal is to find the solution of a given PDE, and we aim to approximate it using a neural network. In the process of doing so, we first have the PDE alongside an SDE of an underlying state with known initial condition. From this, we can get the SDE of the state variable modeled by the PDE using Ito's Lemma. This SDE has a known terminal condition. Once we have a system of FBSDE, we can use the approach described here to find or approximate the solution of the SDE which, in turn, can be used as an approximation for the solution of the PDE we are interested in.

3 Tensor Neural Networks

3.1 Tensor Networks

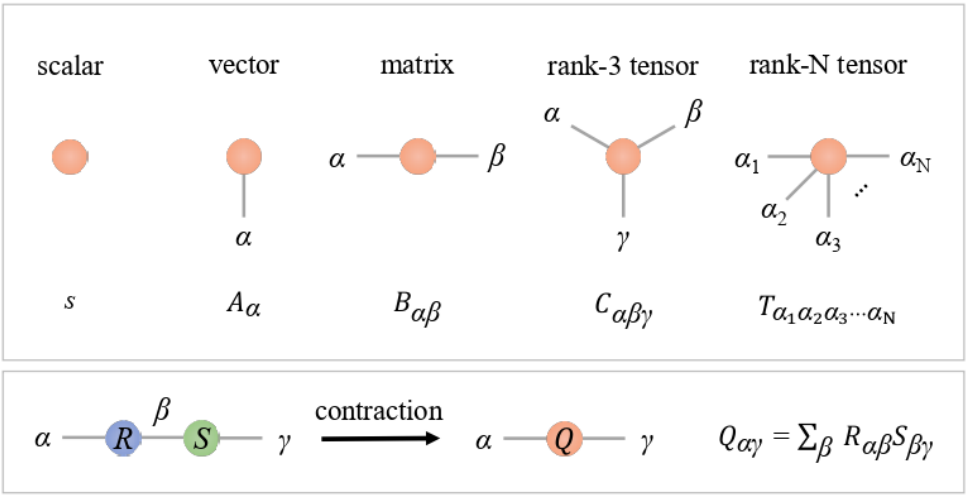


Figure 1: (**upper panel**) Tensor network diagrams. Each rank-n tensor is represented by a shape with n legs attached, each of which represents an index of the tensor. A scalar, vector, matrix and rank-N tensor have 0, 1, 2, N legs attached, respectively. (**lower panel**) Contraction of tensors equivalent to matrix multiplication. Here, R and S tensors are connected with the shared leg β. The resulting tensor (matrix) Q after full contraction has only open legs left.

For our purposes, a tensor is a multi-dimensional array of real/complex numbers, denoted as $T_{\alpha_1\alpha_2\alpha_3\ldots}$, where the indices denote different array dimensions, with the rank of the tensor defined as the number of indices. Tensor network diagrams were introduced as a convenient notation for dealing with the calculation of multiple tensors with multiple indices. As illustrated in Fig. 1, a rank-n tensor is an object with n legs corresponding to its n indices, with each index corresponding to a dimension of the array.

A tensor network is a network of interconnected tensors. By definition, a connected leg between neighboring tensors indicates a shared index to be summed over. The operation of summing over shared indices is also called tensor contraction, and a full contraction will leave a TN with only open (non-contracted) legs. For example, contraction of two rank-2 tensors, i.e., matrices $R_{\alpha\beta}$ and $S_{\beta\gamma}$ along the index β is equivalent to a matrix multiplication. Diagrammatically, the pre-contraction state is shown by connecting the two tensors with their β legs, as depicted in

the lower panel of Fig. 1. Typical one-dimensional TNs include Matrix Product States (MPS) and Matrix Product Operators (MPO) [20] as illustrated in Fig. 2.

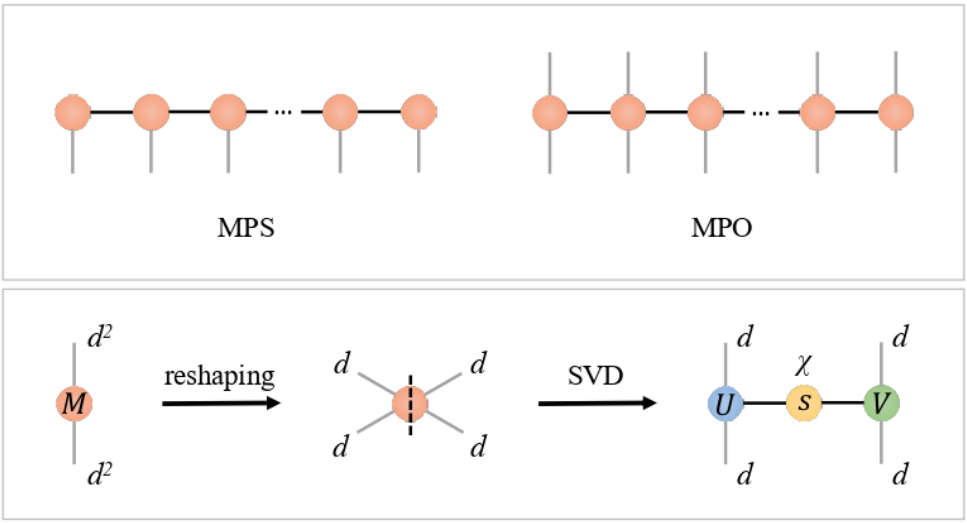


Figure 2: (**upper panel**) Matrix Product State (MPS) and Matrix Product Operator (MPO). (**lower panel**) Tensorizing a matrix into a 2-node MPO by SVD decomposition, producing two isometric tensors U and V, as well as a diagonal matrix S of real and positive singular values.

Generally speaking, MPOs are efficient representations of linear operators, i.e., matrices. In general, a $d^n \times d^n$ matrix can be decomposed (tensorized) into a n-site MPO by a series of consecutive singular value decompositions (SVD). In such a decomposition, at every step there exist at most d^2 singular values. Notably, by discarding sufficiently small singular values we can find an approximated but more efficient MPO representation of the matrix. The lower panel of Fig. 2 shows an example of tensorizing a $d^2 \times d^2$ matrix into a 2-site MPO. The grey legs denote what we call the *physical indices* (i.e., those that correspond to the original tensor), whereas the black line in-between represents what we call a *virtual bond*, with bond dimension χ, corresponding to the number of singular values.

3.2 Tensorizing Neural Networks

A way of tensorizing Neural Networks is to replace the weight matrix of a dense layer by a TN. In particular, we choose an MPO representation of the weight matrix that is analogous to the Tensor-Train format [19], and we call this layer a *TN layer*. This representation, however, is not unique, and is determined by two additional parameters: the MPO bond dimension, and the number of tensors in the MPO. In the simplest case, the MPO may consist of just two tensors only, $\mathbf{W_1}$ and $\mathbf{W_2}$, as shown in Fig. 3. The MPO in the figure has bond dimension χ and physical dimensions d everywhere. The TN layer with such an MPO can be initialized in the same manner as a weight matrix of a dense layer.

The forward pass of the TN layer involves additional operations compared to the one for a dense layer. Here, *we first contract the MPO along the bond index* and then reshape the resulting rank-4 tensor into a matrix as shown in Fig. 3. This matrix is the corresponding weight matrix of a TN layer. The weight matrix can then be multiplied with the input vector. We apply an activation function to the resulting output vector, thereby finishing the forward pass of the TN layer. The weight matrix takes the form

$$\mathbf{W} = \sum_{\alpha=1}^{\chi} (\mathbf{A}_\alpha \otimes \mathbf{B}_\alpha), \quad \mathbf{W} \in \mathbb{R}^{d^2 \times d^2}, \tag{6}$$

where $\mathbf{W}_1 = [\mathbf{A}_1, \mathbf{A}_2, \cdots, \mathbf{A}_\chi]$, $\mathbf{A}_\alpha \in \mathbb{R}^{d \times d}$ and $\mathbf{W}_2 = [\mathbf{B}_1, \mathbf{B}_2, \cdots, \mathbf{B}_\chi]$, $\mathbf{B}_\alpha \in \mathbb{R}^{d \times d}$ are the two rank-3 weight tensors connected by a virtual bond α of dimension χ. The resulting weight matrix $\mathbf{W}$ is of dimension $d^2 \times d^2$, so it contains d^4 elements. Notice that these elements *are not independent* since they come from the TN structure with $2\chi d^2$ trainable parameters. So, if we initialized the MPO with bond dimension $\chi = d^2/2$, we would have the same number of parameters as a dense layer with d^2 neurons. Any choice where $\chi < d^2/2$ will result in a weight matrix

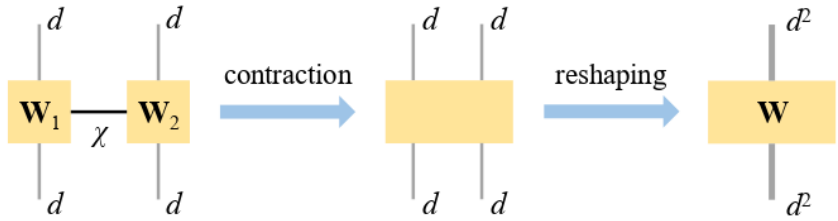


Figure 3: The process of contracting a 2-node MPO and reshaping it into the weight matrix $\mathbf{W}$ in each forward pass.

$\mathbf{W}$ comprising of $d^4 - 2\chi d^2$ fewer parameters than the weight matrix of a dense layer, thus allowing for potential parameter savings. In principle, when $\chi = d^2$, we have sufficient degree of freedom to be able to construct an arbitrary $d^2 \times d^2$ matrix. Thus, we expect that by increasing the bond dimension the TN layer behaves increasingly similar to a dense layer. This is also shown empirically in Section 4.

The existence of Kronecker product in Eq.(6) implies that there is correlation between the matrix elements in $\mathbf{W}$, i.e. each element will be a sum of products of elements of the tensors $\mathbf{A}$ and $\mathbf{B}$. The parameters to be trained are not the matrix elements of the weight matrix, but the elements of the individual tensors of the MPO. This can exhibit interesting training behavior and can lead to faster convergence of the loss function as we show in Section 4.

By implementing the TN layer in this way and with a ML library which supports automatic differentiation such as TensorFlow or PyTorch, one can optimize the MPO weights in a similar fashion as those of dense layers in DNN and train the TNN. As an alternative, one could work with an explicit TN layer without contraction of the MPO, including tensor optimizations as in other TN algorithms (e.g., variational), provided one can find a way to decompose the activation function. We observe, however, that for most interesting NN structures, we do not actually need this option. Additionally, the TNN structure is not limited only to a single TN layer but can further be extended to any desired number of TN layers or a combination of dense and TN layers. This provides a flexibility of designing TNN architectures which is favorable for the problems of interest.

4 Black-Scholes-Barenblatt Equation in 10 dimensions

We test our approach on an application in mathematical finance where we approximate the solution of a 10-dimensional Black-Scholes-Barenblatt equation to price a European-style option. This is primarily motivated by the pioneering work of using deep learning to solve high-dimensional PDEs [23, 7, 16].

We aim to solve the following PDE:

$$u_t = -\frac{1}{2}\mathrm{Tr}\left(\sigma^2 \mathrm{diag}(X_t^2)\mathcal{D}^2 u\right) + r(u - (\mathcal{D}u)'x), \tag{7}$$

with terminal condition $u(T, x) = \|x\|^2$. The explicit solution for this equation is

$$u(t, x) = \exp((r + \sigma^2)(T - t))\|x\|^2, \tag{8}$$

which can be used to test the accuracy of the proposed algorithm. Using the approaches from Section 2 and the Feynman-Kac formalism from Appendix A, this can be re-casted into a system of forward-backward stochastic differential equations:

$$\begin{aligned} dX_t &= \sigma \mathrm{diag}(X_t) dW_t, \quad t \in [0, T] \\ X_0 &= \epsilon \\ dY_t &= r(Y_t - Z_t' X_t)dt + \sigma Z_t' \mathrm{diag}(X_t) dW_t, \quad t \in [0, T) \\ Y_T &= ||X_T||^2 \end{aligned} \tag{9}$$

where $\epsilon = (1, 1, 1, \ldots, 1) \in \mathbb{R}^{10}$, $T = 1$, $\sigma = 0.4$, $r = 0.05$. W_t is a vector-valued Brownian motion, whereas Y_t and Z_t represent $u(t, X_t)$ and $Du(t, X_t)$. Furthermore, we partition the time domain $[0, T]$ into $N = 50$ equally spaced intervals. For loss, instead of using mean squared error (MSE) which is classically used for regression problems, we use log-cosh loss which helps in speeding up the convergence as compared to the work in Ref. [23] and which is of the form $\frac{1}{N}\sum_{i=1}^{N} \ln(\cosh(\hat{y}_i - y_i))$. We further sketch out the details of this loss function and its advantages in Appendix B. To optimize our model weights, we use Adam Optimizer with batch of size 100 and a fixed learning rate of 10^{-3}. Given the simplicity of the payoff structure, for all our experiments, we use a 2-hidden layer architecture. For simplicity, we only construct TN layers that are symmetric in each input and each output dimension. In practice, we choose the first layer in our NN to be a dense layer with neurons that match the input shape of the second TN layer. That is, a DNN(x, y) corresponds to a two-layer dense network with x neurons in layer 1 and y neurons in layer 2. On the other hand, a TNN(x) corresponds to a two layer TNN architecture with the first being a dense layer with x neurons and the second layer a TN layer with x neurons.

We described in Section 3 how TNN can compress dense layers, which was experimentally tested in Ref. [19]. However, this compression is only relevant if no DNN with the same parameter count can achieve the same accuracy and training performance.

In Fig. 4, we see the loss behavior and the option price at t = 0 for three different architectures, TNN(16), DNN(16,16) and DNN(6,35). Note that, in comparison with TNN(16), DNN(16,16) has the same number of neurons but more parameters. Whereas, DNN(6,35) has the same number of parameters but different number of neurons. All three architectures achieve the same accuracy level upon convergence. So, although TNN(16) is achieving the same accuracy as DNN(16,16) with fewer parameters, we find DNN(6,35) to be equally good in terms of accuracy and number of parameters. Hence, the number of parameters may not be used as a measure of compression without considering alternative DNN architectures with same parameter counts, which is a major drawback in the experiments performed in Ref. [19].

Moreover, we see in our experiments that the architectures differ in convergence speed. DNN(6,35) converges fastest among all the DNN architectures with the same parameter count as that of TNN(16). However, we observe that the TNN architecture converges even faster than DNN(6,35). It also converges faster than DNN(16,16). In summary, TNN not only allows for memory savings with respect to DNN for the same number of neurons, but also for faster convergence for the same number of parameters and neurons.

To get a better understanding of the advantages and behavior of TNN after taking various hyperparameters into account, we further implemented the following three experiments: (a) testing convergence speed for TNN with increasing bond dimension, (b) testing convergence speed for TNN with increasing number of neurons, and (c) finding the best matching DNN architecture (in terms of convergence speed and accuracy) and its parameter count as compared to a TNN.

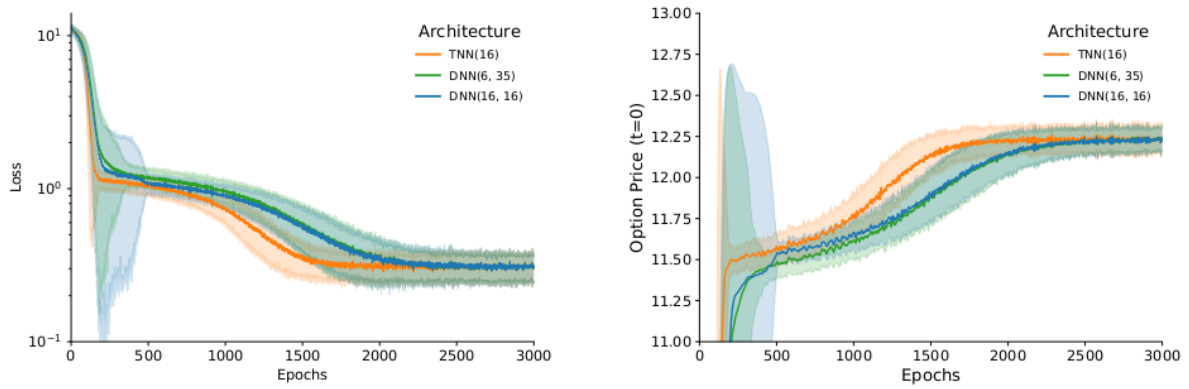


Figure 4: (**left panel**) Training loss and (**right panel**) option price at t_0 over epochs of TNN(16) bond dimension 4 case (orange) and the corresponding best performing DNN with the same number of parameters (in this case, 353). The plots indicate resulting mean $\pm$ standard deviation from 100 runs.

Let us discuss experiment (a), where we analyzed the effect of the bond dimension to the convergence behavior. We choose two TN architectures, TN(16) and TN(64) and vary their bond dimension χ. For each χ we calculate the number of parameters in the network and construct all possible two-layer DNN with the same parameter count. We train the networks until convergence (convergence criteria is described in Appendix B) and show the results in Fig. 5. We observe that almost all TNN architectures train faster than their DNN counterparts. The convergence gap is the largest for TNN(16) for $\chi = 4$ with 26.9% fewer epochs to converge compared to the best performing DNN with the same parameter count. Furthermore, we see that with increasing bond dimension, the convergence gap diminishes, which is a signature that the corresponding MPO is approaching a dense layer and therefore the performance of a DNN as seen in our discussion in Section 3. An MPO with sufficiently large bond dimension can approximate any arbitrary matrix to an arbitrary degree. Hence, a TNN with a large bond dimension should exhibit a weight matrix in the forward pass that is very much comparable to an equally sized weight matrix of a DNN.

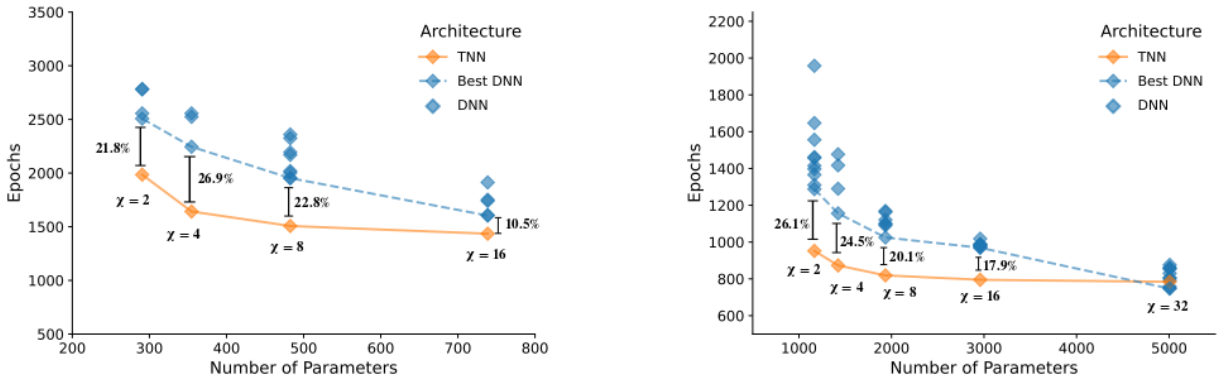


Figure 5: Epoch for loss convergence for TNN(16) (**left panel**) and TNN(64) (**right panel**) over different bond dimensions. Each TNN is shown along with all DNNs of same number of parameters. The plots indicate results from 100 runs for the architectures reaching accuracy within 1% of the true solution.

Model complexity is another important factor when discussing the advantages of different architectures. We chose not to go deeper than two-layer networks due to the simplicity of the PDE at hand. However, we would like to know how TNN behaves if we construct wider networks. To this end, we increase the number of neurons in experiment (b). We choose $\chi = 2$ and $\chi = 4$ for the TNN we analyse and use the same convergence criteria as in experiment (a). The results are shown in Fig. 6 for TNN(16), TNN(64), TNN(144) and TNN(256). In all cases, TNN is trained faster than its best DNN counterparts. We find the largest convergence gap of 32.8% for TNN(144) with $\chi = 4$.

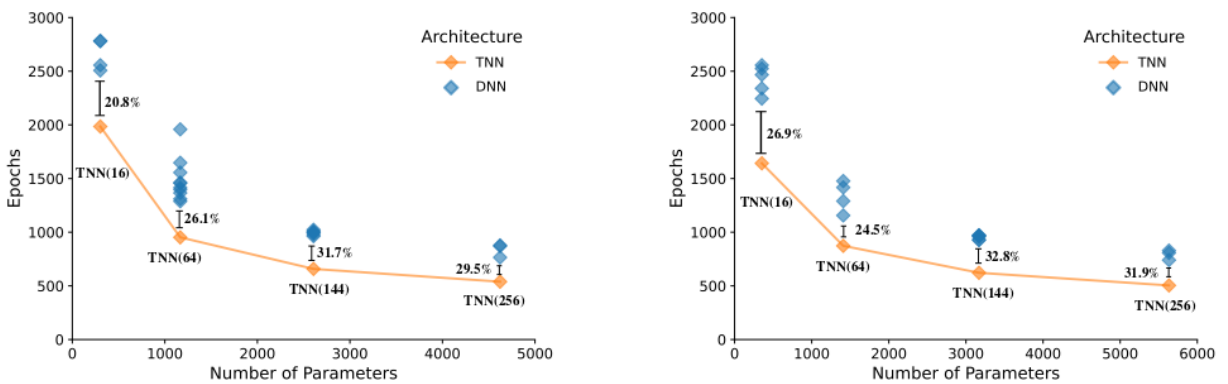


Figure 6: Epoch for loss convergence for TNN of bond dimension 2 (**left panel**) and 4 (**right panel**) with varying number of layer width (TNN(16), TNN(64), TNN(144) and TNN(256)). Each TNN is shown along with all DNNs of same number of parameters. The plots indicate results from 100 runs for the architectures reaching accuracy within 1% of the true solution.

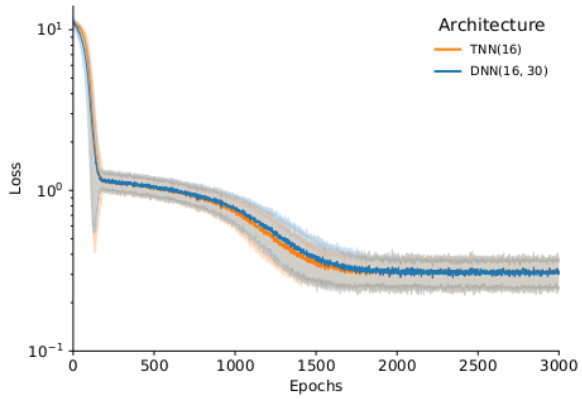


Figure 7: Loss functions for TNN (orange) with bond dimension 4 and its best matching DNN (blue). The TNN has 353 parameters whereas the DNN has 1057 parameters. The plots indicate resulting mean $\pm$ standard deviation from 100 runs

Finally, in experiment (c), we analyse the parameter savings of the TNN for the example presented in Fig. 4. One way of quantifying this is to find the smallest DNN architecture that matches the loss signature of TNN, i.e., one that has the same training speed and accuracy. Fig. 7 shows such an example, where we compare a DNN architecture with 1057 parameters to that of a TNN with 353 parameters. We conclude that *with DNN we almost need three times the number of parameters to match the TNN's performance*. This parameter saving of 66% is indeed a significant advantage for TNN.

To consolidate the TNN advantage observed in the plots from this section, we also tested TNN on the Hamilton-Jacobi-Bellman equation, which can be found in Appendix B.

5 Conclusions and Outlook

We have shown how we can leverage TNN to solve high-dimensional parabolic PDEs. Furthermore, we addressed some of the shortcomings in the existing literature when quantifying the advantages of TNN by analyzing parameter savings and convergence speed. Empirically, we demonstrated that TNN provides significant parameter savings as compared to a DNN while attaining the same accuracy. We further illustrated that TNN achieves speedup in training by comparing them against entire families of DNN architectures with similar parameter count. The methodology described in this paper can be used to improve training from a memory and speed perspective for a wide variety of problems in machine learning. Quantifying the complexity of a problem and adapting the methodology presented to problems where this approach can provide a significant edge, can be an interesting avenue for future work.

Acknowledgements - We acknowledge regular fruitful discussions with the technical teams both at Crédit Agricole and Multiverse Computing.

References

[1] A. M. Aboussalah, M.-J. Kwon, R. G. Patel, C. Chi, and C.-G. Lee. Don't overfit the history – recursive time series data augmentation. *arXiv preprint arXiv:2207.02891*, 2022.

[2] F. Antonelli. Backward-forward stochastic differential equations. *Annals of Applied Probability*, pages 777–793, 1993.

[3] A. Bhatia, M. K. Saggi, A. Kumar, and S. Jain. Matrix product state–based quantum classifier. *Neural computation*, 31(7):1499–1517, 2019.

[4] T.-D. Bradley, E. M. Stoudenmire, and J. Terilla. Modeling sequences with quantum states: A look under the hood. *Machine Learning: Science and Technology*, 2020.

[5] S. Cheng, L. Wang, T. Xiang, and P. Zhang. Tree tensor networks for generative modeling. *Physical Review B*, 99:155131, 2019.

[6] P. Cheridito, H. M. Soner, T. Nizar, and N. Victoir. Second-order backward stochastic differential equations and fully nonlinear parabolic PDEs. *Communications on Pure and Applied Mathematics*, 60(7):1081–1110, nov 2006.

[7] B. Christian, E. Weinan, and J. Arnulf. Machine learning approximation algorithms for high-dimensional fully nonlinear partial differential equations and second-order backward stochastic differential equations. *Journal of Nonlinear Science*, 29(4):1563–1619, jan 2019.

[8] F. Delarue and S. Menozzi. A forward-backward stochastic algorithm for quasi-linear PDEs. *The Annals of Applied Probability*, pages 140–184, 2006.

[9] M. Denil, B. Shakibi, L. Dinh, M. A. Ranzato, and N. de Freitas. Predicting parameters in deep learning. *Advances in Neural Information Processing Systems*, 26:2148–2156, 2013.

[10] S. Edwin. Learning relevant features of data with multi-scale tensor networks. *Quantum Science and Technology*, 3(3):034003, 2018.

[11] S. Edwin and D. J. Schwab. Supervised learning with tensor networks. *Advances in Neural Information Processing Systems 29*, pages 4799–4807, 2016.

[12] S. Efthymiou, J. Hidary, and S. Leichenauer. Tensor network for machine learning. *arXiv preprint arXiv:1906.06329*, 2019.

[13] I. Glasser, N. Pancotti, and J. I. Cirac. Supervised learning with generalized tensor networks. *arXiv preprint arXiv:1806.05964*, 2018.

[14] I. Glasser, N. Pancotti, and J. I. Cirac. From probabilistic graphical models to generalized tensor networks for supervised learning. *IEEE Access*, 8:68169–68182, 2020.

[15] Z.-Y. Han, J. Wang, H. Fan, L. Wang, and P. Zhang. Unsupervised generative modeling using matrix product states. *Physical Review X*, 8:031012, 2018.

[16] H. Jiequn, J. Arnulf, and E. Weinan. Solving high-dimensional partial differential equations using deep learning. *Proceedings of the National Academy of Sciences*, 115(34):8505–8510, aug 2018.

[17] D. Liu, S.-J. Ran, P. Wittek, C. Peng, R. B. García, G. Su, and M. Lewenstein. Machine learning by unitary tensor network of hierarchical tree structure. *New Journal of Physics*, 21(7):073059, 2019.

[18] J. Ma, P. Protter, and J. Yong. Solving forward-backward stochastic dif- ferential equations explicitly: a four step scheme. *Probability theory and related fields*, 98:339–359, 1994.

[19] A. Novikov, D. Podoprikhin, A. Osokin, and D. P. Vetrov. Tensorizing neural networks. *Advances in Neural Information Processing Systems*, 28, 2015.

[20] R. Orús. A practical introduction to tensor networks: Matrix product states and projected entangled pair states. *Annals of Physics*, 349:117–158, 2014.

[21] I. V. Oseledets. Tensor-train decomposition. *SIAM Journal on Scientific Computing*, 33(5):2295–2317, 2011.

[22] E. Pardoux and S. Tang. Forward-backward stochastic differential equations and quasilinear parabolic PDEs. *Probability theory and related fields*, 114:123–150, 1999.

[23] M. Raissi. Forward-backward stochastic neural networks: Deep learning of high-dimensional partial differential equations. *arXiv preprint arXiv:1804.07010v1*, 2018.

[24] M. Raissi, P. Perdikaris, and G. E. Karniadakis. Physics informed deep learning (part i): Data-driven solutions of nonlinear partial differential equations. *arXiv preprint arXiv:1711.10561*, 2017.

[25] M. Raissi, P. Perdikaris, and G. E. Karniadakis. Physics informed deep learning (part ii): Data-driven discovery of nonlinear partial differential equations. *arXiv preprint arXiv:1711.10566*, 2017.

[26] T. N. Sainath, B. Kingsbury, V. Sindhwani, E. Arisoy, and B. Ramabhadran. Low-rank matrix factorization for deep neural network training with high-dimensional output targets. *International Conference on Acoustics, Speech and Signal Processing*, pages 6655–6659, 2013.

[27] K. Simonyan and A. Zisserman. Very deep convolutional networks for large-scale image recognition. *International Conference on Learning Representations (ICLR)*, 2015.

[28] Z.-Z. Sun, C. Peng, D. Liu, S.-J. Ran, and G. Su. Generative tensor network classification model for supervised machine learning. *Physical Review B*, 101:075135, 2020.

[29] G. Vidal. Efficient simulation of one-dimensional quantum many-body systems. *Phys. Rev. Lett.*, 93:040502, Jul 2004.

[30] E. Weinan, H. Jiequn, and J. Arnulf. Deep learning-based numerical methods for high-dimensional parabolic partial differential equations and backward stochastic differential equations. *Communications in Mathematics and Statistics*, 5(4):349–380, nov 2017.

[31] S. R. White. Density matrix formulation for quantum renormalization groups. *Phys. Rev. Lett.*, 69:2863–2866, Nov 1992.

[32] J. Xue, J. Li, and Y. Gong. Restructuring of deep neural network acoustic models with singular value decomposition. *Interspeech, 2013*, pages 2365–2369, 2013.

Appendix

A Feynman-Kac Representation

Let $\mu : \mathbb{R}_+ \times \mathbb{R}^d \to \mathbb{R}^d$, $\sigma : \mathbb{R}_+ \times \mathbb{R}^d \to \mathbb{R}^{d\times m}$, $V : \mathbb{R}_+ \times \mathbb{R}^d \to \mathbb{R}$ and $f : \mathbb{R}^d \to \mathbb{R}$ be sufficiently regular functions, fix $T > 0$ and suppose $u \in C^{1,2}([0,T] \times \mathbb{R}^d \to \mathbb{R})$ is the unique solution to the parabolic differential equation

$$\begin{aligned} \frac{\partial u}{\partial t} + Au &= Vu \\ u(T,x) &= f(x) \end{aligned} \tag{10}$$

with generator defined by

$$Ag(t,x) = \sum_{i=1}^{d} \mu_i(t,x)\frac{\partial g}{\partial x_i}(t,x) + \frac{1}{2}\sum_{i,j=1}^{d} \left(\sigma\sigma^T\right)_{ij}(t,x)\frac{\partial^2 g}{\partial x_i \partial x_j}(t,x) \tag{11}$$

for $g \in C^{1,2}([0,T] \times \mathbb{R}^d \to \mathbb{R})$. If Y is the $\mathbb{R}^d$-valued stochastic process satisfying the stochastic differential equation

$$dY_t = \mu(t,Y_t)dt + \sigma(t,Y_t)dB_t$$

for some m-dimensional Brownian motion B defined on a probability space $(\Omega, \mathcal{F}, \mathbb{P})$, then u admits the stochastic representation

$$u(t,x) = \mathbb{E}^{\mathbb{P}}\left[e^{-\int_t^T V(s,X_s)ds} f(Y_T) \mid Y_t = x\right]$$

for all $(t,x) \in [0,T] \times \mathbb{R}^d$.

B Further experiments

B.1 Hamilton-Jacobi-Bellman Equation

Dynamic Decision Making (DDM) lies at the heart of a lot of industrial problems spanning from inventory control and robotics to finance. Integral to DDM is the difference equation called Bellman equation which is an optimality condition associated with dynamic programming, an approach which has been fundamental in solving DDM problems recursively. This Bellman equation in continuous form leads us to the Hamilton-Jacobi-Bellman (HJB) PDE which is an indispensable tool in the area of stochastic control problems. Here, we consider a 100-dimensional HJB equation as shown in Ref. [23] which looks as follows:

$$u_t = -\mathrm{Tr}\left(\mathcal{D}^2 u\right) + \|\mathcal{D}u\|^2, \tag{12}$$

with terminal condition $u(T,x) = \ln\left(0.5(1+\|x\|^2)\right)$. The explicit solution for this equation is

$$u(t,x) = -\ln\left(\mathbb{E}\left[\exp(-g(x+\sqrt{2}W_{T-t}))\right]\right), \tag{13}$$

which can be used to test the accuracy of the proposed algorithm. However, due to the presence of the expectation operator in the explicit solution, we can only approximate the exact solution. To do so, we use 10^5 samples to approximate the exact solution as done in Ref. [23]. Using Feynman-Kac as shown in Section 2, this can be recasted into a system of forward-backward stochastic differential equations as follows:

$$\begin{aligned} dX_t &= \sigma dW_t, \quad t \in [0,T] \\ X_0 &= \epsilon \\ dY_t &= \|Z_t\|^2 dt + \sigma Z_t' dW_t, \quad t \in [0,T) \\ Y_T &= \ln\left(0.5\left(1+\|X_T\|^2\right)\right), \end{aligned} \tag{14}$$

where $\epsilon = (0,0,0,\ldots,0) \in \mathbb{R}^{100}$, $T = 1$, $\sigma = \sqrt{2}$. Furthermore, we partition the time domain $[0,T]$ into $N = 50$ equally spaced intervals. For loss, here too, instead of using mean squared error (MSE) which is classically used for regression problems, we use log-cosh loss which helps in speeding up the convergence as compared to the work in Ref. [23] and which is of the form $\frac{1}{N}\sum_{i=1}^{N}\ln(\cosh(\hat{y}_i - y_i))$. To optimize our model weights, we use Adam Optimizer with batch of size 100 and a fixed learning rate of 10^{-3}.

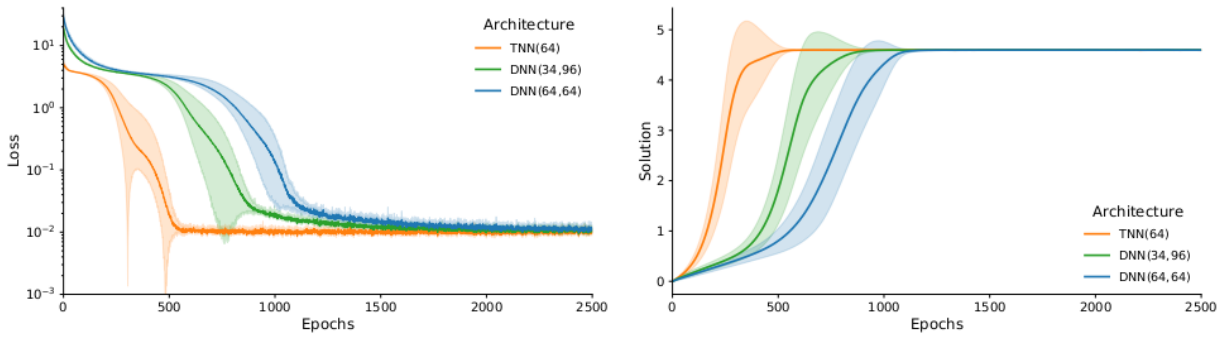


Figure 8: (**left panel**) Training loss and (**right panel**) Solution for 100-d HJB equation at t_0 over epochs for TNN(64) with bond dimension 2 (orange), the corresponding best DNN with equivalent parameters (blue) and the DNN with equivalent neurons (green).

As shown in Fig. 8, a TNN of 64 units and bond dimension 2 clearly outperforms all DNNs with the same parameter count in terms of convergence speed, thereby showing TNN's clear advantage (in this plot, we plot DNN(34,96) which has the best performance among the DNNs with the same parameter count as TNN). Here, the advantage is even more observable, with TNN converging in 66% fewer epochs (which is 63% savings in time taken to converge). In the plot, we also show the comparison of TNN with a DNN with same number of neurons, i.e., DNN(64,64). Despite the DNN having same number of neurons and rather more parameters, TNN clearly outperforms it.

B.2 Same Initialization Magnitude

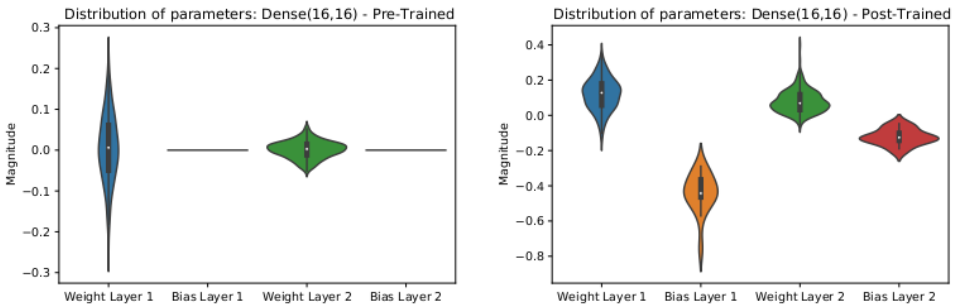


Figure 9: Pre-trained weights (left) and Post-trained weights (right) for a dense architecture with (16,16) neurons in the two hidden layers

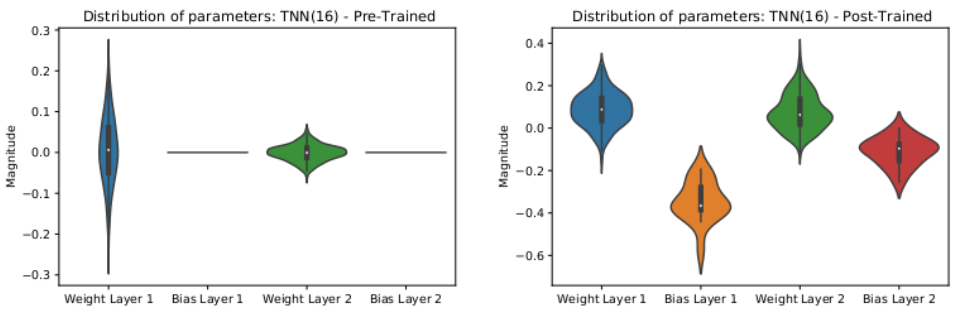


Figure 10: Pre-trained weights (left) and Post-trained weights (right) for a TNN architecture with bond dimension 8 and 2 tensors. This architecture has same number of parameters as the one in Fig. 9

In Section 3, we saw that, for a TNN layer, we first initialize a set of tensors and then contract them to a shape that is equivalent to that of a dense layer. In doing so, we realize that if the tensors are initialized in a similar way as the

weight matrix of a DNN layer, the magnitude of initialized weights is different when comparing the contracted tensor of a TNN layer with a weight matrix from a DNN layer with same size. This can often lead to different training trajectories. However, to ensure that the resulting advantage stems from the underlying structure of TNN and not magnitude of initialized weights, we also initialize them in a way to ensure similar magnitude. Here, we start by setting up weights in a way that the distribution of the pre-trained weights of a DNN layer and that of an equivalent contracted TNN layer with same number of parameters are similar as shown in Figs. 9 and 10. Despite having the same pre-trained weights, we still observe the TNN advantage as Fig. 4, which can also be attributed to different post-trained weight distribution.

B.3 Hybrid Loss Function

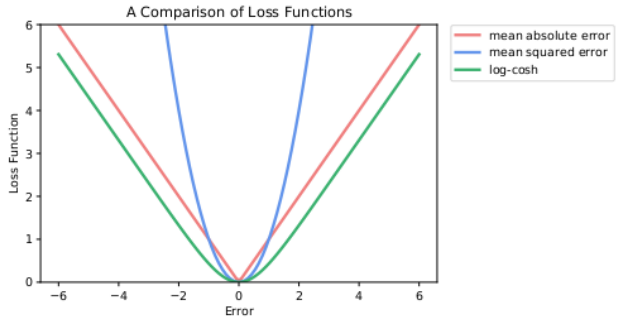


Figure 11: Comparison of Mean Squared Error (MSE), Mean Absolute Error (MAE) and Hybrid loss for regression-type problems

For our experiments in this paper, we use a hybrid loss function over MSE due to its empirically enhanced speed of convergence as observed in our experiments. We use log-cosh loss of the form $\frac{1}{N}\sum_{i=1}^{N}\ln(\cosh(\hat{y_i}-y_i))$. The function $\ln(\cosh(x))$ for small x turns out to be $\frac{x^2}{2}$, whereas for large x this becomes $|x|-\ln(2)$. Comparing it to MSE, it looks as in Fig. 11. However, due to empirical results, we only apply this to the terminal conditions. As a result, our loss function changes from

$$\min_{\theta}\Bigg(\sum_{m=1}^{M}\sum_{n=0}^{N-1}|Y_{n+1}^{m}(\theta)-Y_{n}^{m}(\theta)-\varphi(t_n,X_n^m,Y_n^m(\theta),Z_n^m(\theta))\Delta t_n - (Z_n^m(\theta))^T\sigma(t_n,X_n^m,Y_n^m(\theta))\Delta W_n^m|^2+\sum_{m=1}^{M}(Y_N^m(\theta)-g(X_N^m))^2\Bigg) \quad (15)$$

to

$$\min_{\theta}\Bigg(\sum_{m=1}^{M}\sum_{n=0}^{N-1}|Y_{n+1}^{m}(\theta)-Y_{n}^{m}(\theta)-\varphi(t_n,X_n^m,Y_n^m(\theta),Z_n^m(\theta))\Delta t_n - (Z_n^m(\theta))^T\sigma(t_n,X_n^m,Y_n^m(\theta))\Delta W_n^m|^2+\frac{1}{M}\sum_{m=1}^{M}\ln(\cosh(Y_N^m(\theta)-g(X_N^m)))\Bigg) \quad (16)$$

B.4 Convergence Test

In this paper, we run experiments for a large number of epochs to ensure guaranteed convergence for both, DNN and TNN. For defining convergence for loss, we use the criteria in the pseudocode 1 based on smoothed time series of mean loss [1].

Algorithm 1 Convergence Test

Input: Observed Time Series of Mean Loss for 100 runs over 3000 epochs L=$\{l_1, l_2, \ldots, l_{3000}\}$, window size($w$), batch size($b$), threshold($h$), tolerance($t$)

Smoothing: Initialize α and obtain Smoothed Time Series ($\hat{L}$) with same starting point l_1 and hence $\hat{L} = \{\hat{l}_1, \hat{l}_2, \ldots, \hat{l}_{3000}\}$ where $\hat{l}_1 = l_1$ and $\hat{l}_i = \alpha\hat{l}_{i-1} + (1-\alpha)l_i$ for $i > 1$

for i=1 to epochs-w **do**

$Window = \hat{L}[i : i+w]$
$Diff$ = $|mean(Window[: b])|$ - $|mean(Window[w-b :])|$
▷ Difference between the loss trajectory at the start and the end of the window for a batch of entries

if Entries($Window$) $< h$ and $Diff < t$ **then**
return i
▷ Ensure all the elements in window are less than a threshold and $Diff$ is less than a tolerance
else
continue
end if
epochs
end for